

Testatblock 1 (Versuche 1 bis 5)

Versuch 1

Fragen:

1. Wozu dient der Debugger? (Fachwissen, leicht)
2. Was ist ein Haltepunkt und wozu wird er benutzt? (Fachwissen, leicht)
3. Wo liegt der Unterschied zwischen Einzelschritt (Step Into) und Prozedurschritt (Step Over)? (Fachwissen, leicht)
4. Welche Standard Datentypen in C++ kennen Sie? Welche komplexen Datenstrukturen kennen Sie? (Fachwissen, leicht)
5. Was wird als Typcasting oder Casting bezeichnet? Was kann hierbei ein Problem sein? (Verständnis, mittel)
6. Wie lassen sich structs kopieren? (Fachwissen, leicht)
7. Wie sieht der Zugriff auf ein Array aus? Wie ist er bei mehrdimensionalen Arrays? (Fachwissen, leicht)
8. Was versteht man unter dem Gültigkeitsbereich einer Variablen? (Fachwissen, leicht)
9. Welche Möglichkeiten bestehen, Zwischenergebnisse einer Rechnung zu speichern? (Fachwissen, leicht)
10. Wie wird ein Buchstabe im Speicher des Computers dargestellt? (Fachwissen, leicht)
11. Was ist der Unterschied in der Darstellung von einer *char* 1 und einer *int* 1 im Speicher? (Fachwissen, mittel)
12. Was passiert, wenn man eine *char* 1 als *int* castet? (Verständnis, schwierig)
13. Was passiert, wenn man eine *char* 1 als *int* castet , + 1 rechnet und anschließend zurück nach *char* castet? (Verständnis, schwierig)
14. Wie wird ein Array im Speicher abgelegt? (Fachwissen, leicht)
15. Was ist der Unterschied zwischen char arrays und strings? (Fachwissen, leicht)
16. Sie wollen einen Benutzernamen speichern. Wie würden Sie dies machen? (Anwenden, leicht)
17. Was ist der Unterschied zwischen Präfix und Postfix Inkrement/Dekrement? Wann verwendet man welches? (Verständnis, schwierig)
18. Was muss beachtet werden, wenn zwei Variablen, deren Werte nahe an der Grenze ihres Wertebereichs liegen, addiert/multipliziert werden? (Auch wenn das Ergebnis in eine Variable mit dem nächst größeren Datentyp gespeichert wird!) (Verständnis, Schwierig)
19. Wie können Funktionszeichen wie \oder " in der Standardausgabe ausgegeben werden? (Fachwissen, leicht)
20. Wozu dient der Aufzählungstyp enum? Wie wird er verwendet? (Fachwissen, schwierig)
21. Was bedeutet implizites/explizites typecasting? (Fachwissen, schwierig)

Antworten:

1. Der Debugger kann zum Suchen von semantischen/inhaltlichen Programmfehlern genutzt werden, die nicht vom Compiler gefunden werden können.
2. Hält das Programm während der Ausführung an der gewünschten Stelle an. Kann zur gezielten Fehlersuche an bestimmten Stellen genutzt werden.
3. Step Into: Der Debugger springt in die auszuführende Funktion hinein, dort kann dann weiter gedebuggt werden. Sollte die ausgewählte Zeile keine Funktion darstellen, in die hinein gesprungen werden kann, so wird die Zeile direkt ausgeführt.
Step Over: Die ausgewählte Zeile wird als ganzes ausgeführt, unabhängig davon, ob weitere Funktionen in der Zeile aufgerufen werden.
4. bool, char, int, float, double, long, unsigned int, unsigned float, unsigned double, unsigned long, struct, enum
5. Gezielte Umwandlung von Datentypen untereinander. Gerade bei Divisionen kann es zu nicht gewünschten Ergebnissen führen, wenn zwei Zahlen unterschiedlichen Datentyps dividiert werden.
6. Structs können geschlossen kopiert werden. Elementweises kopieren ist auch möglich, jedoch deutlich aufwändiger.
7. Eindimensionales Array: `int x[3] = {1, 2, 3};`
`x[0] = 1 , x[1] = 2 , x[2] = 3`
Mehrdimensionales Array: `int y[2][3] = {{1, 2, 3}, {4, 5, 6}};`
`y[0][0] = 1 , y[0][1] = 2 , y[0][2] = 3 , y[1][0] = 4 , y[1][1] = 5 , y[1][2] = 6`
8. Der Bereich, in dem auf Variablen zugegriffen werden kann und wo diese gültig sind. Für lokale Variablen ist dies meist der Bereich zwischen geschweiften Klammern " ".
9. In lokalen Variablen
10. als char
11. Bei einer char '1' wird die Zeichenkette 1 gespeichert, laut ASCII Tabelle entspricht dies einer 49. Eine int '1' wird als 1 im Speicher abgelegt.
12. Der Speicherbereich des char wird als int interpretiert, dies entspricht einer 1.
13. Da '1' und '2' in der ASCII Tabelle direkt aufeinander folgen, erhält man eine '2' als Ergebnis.
14. Die Array-Elemente werden in aufeinander folgenden Speicherstellen abgelegt.
15. char array haben eine vorher definierte Größe/Länge. Bei Strings handelt es sich um eine Klasse, die Größe/Länge der Zeichenkette muss vorher nicht bekannt sein und kann während der Laufzeit variiert werden.
16. Am besten mit Hilfe eines Strings, aber auch ein char array mit einer ausreichenden Länge ist denkbar.
17. Präfix: erst Addition, dann Zuweisung Postfix: erst Zuweisung, dann Addition (Addition betrifft in dem Fall nur Variable auf die Ink/Dek angewendet wurde)
18. `int a = 230;`
`long b = 8 * a;`
liefert nicht richtiges Ergebnis!
Richtig wäre z.B.: `long b = 8L * a;`
19. Durch Maskierung mit \
20. Um mehreren Variablen mit inkrementierten Werten zu initialisieren Bspw. praktisch bei Pin/Registerdefinitionen von Mikrocontrollern (const enum)
21. implizit: Compiler castet selbständig bei Operationen in passenden Datentypen. explizit: explizite Angabe in welchen Typ der Wert gecastet werden soll.

Versuch 2

Fragen:

1. Was sind Funktionen und wozu werden Sie benutzt? (Fachwissen, leicht)
2. Wie werden Funktionen gekennzeichnet, die einen Wert zurückgeben? (Fachwissen, leicht)
3. Wie wird eine Funktion gekennzeichnet, die keinen Wert zurück gibt? (Fachwissen, leicht)
4. Was versteht man unter der Deklaration einer Funktion und was unter der Definition und was unter der Implementierung? (Fachwissen, mittel)
5. Erläutern Sie den Unterschied zwischen Call-by-value und Call-by-reference? (Verständnis, mittel)
6. Wie werden Felder an eine Funktion übergeben? (Fachwissen, leicht)
7. Welche Möglichkeiten gibt es in C/C++, um innerhalb des Programms zu verzweigen? (Fachwissen, leicht)
8. Welche Möglichkeiten gibt es in C/C++, um Codeausschnitte mehrfach zu durchlaufen? (Fachwissen, leicht)
9. Wozu dient die break Anweisung? (Fachwissen, leicht)
10. Wozu dient die continue Anweisung? (Fachwissen, leicht)
11. Was ist mit dem Fall-Through bei einer switch Anweisung gemeint? (Fachwissen, leicht)
12. Was unterscheidet die while von der do-while Schleife? (Fachwissen, leicht)
13. Was sind Zeiger und Referenzen? Wo liegt der Unterschied? (Verständnis, schwierig)
14. Was ist Rekursion? (Fachwissen, leicht)
15. Wieso unterscheiden sich die iterative und rekursive Berechnung der Fibonacci-Zahlen in der Laufzeit? (Verständnis, mittel)
16. Wie funktioniert die Caesar-Verschlüsselung? (Verständnis, leicht)
17. Wo liegt der Unterschied zwischen automatischem, statischem und dynamischen Speicher? (Verständnis, schwierig)
18. Wie kann man Fall-Through bei einer switch Anweisung vermeiden? (Anwenden, mittel)
19. Welcher Art muss das Argument bei der Switch-Case-Struktur sein? (Fachwissen, mittel)
20. Wie sieht die Signatur für eine Funktion aus, die ein Array übergeben bekommt? (Anwenden, mittel)
21. Wozu dient die return Anweisung? (Fachwissen, leicht)

Antworten:

1. Funktionen kapseln bestimmte Teile eines Programms. Sie stellen meist immer wiederkehrende Funktionalität zur Verfügung. Typische Beispiele wären mathematische Berechnungen, set und get-Funktionalität, aber auch komplexe Algorithmen lassen sich in Funktionen unterbringen. Funktionen steigern die Lesbarkeit des Programms deutlich.
2. Durch Angabe des Rückgabewertes vor dem Funktionsnamen (Beispiel: `double foo()` oder `bool foo()`)
3. Durch Angabe von `void` (Beispiel: `void foo()`)
4. Deklaration: *Bekannt machen der Funktion*. Signatur im header-File, also Funktionsname, Typ der Parameter und Rückgabewert.
Definition: Funktionalität der Funktion, eigentliche Implementierung des gewünschten Verhaltens. Implementierung: siehe Definition
5. Call-by-Value: Es wird eine Kopie der Variable beim Aufruf an die Funktion übergeben. Call-by-Reference: Es wird eine Referenz übergeben. Dies hat zur Folge, dass Änderungen der Variablen auch außerhalb der Funktion sichtbar sind. ACHTUNG: Streng genommen handelt es sich bei der Übergabe eines Zeigers um Call-by-Value, laut Skript soll aber nur verstanden werden, dass Änderungen an der Variable auch außerhalb der Funktion sichtbar sind. Im Skript ist die Übergabe von Zeigern auch Call by Referenz)
6. Man muss die Dimensionen der Felder angeben (Beispiel: `sortieren(int x[10]);`)
7. if-else Konstrukte sowie switch Anweisungen
8. for-Schleifen, while-Schleifen, do-while-Schleifen
9. Vorzeitiges Verlassen der Schleife
10. Vorzeitiges Verlassen der aktuellen Iteration und sofortiges Springen zur nächsten Iteration der Schleife
11. *Durchfallen* durch alle cases der Switch Anweisung (Bei den case-Labels handelt es sich lediglich um Einsprungsmarken)
12. Bei einer do-while Schleife wird mindestens eine Iteration durchgeführt, dies muss für while Schleifen nicht der Fall sein.
13. Referenzen sind Aliasnamen für Variablen und müssen bei ihrer Erzeugung an Variablen gebunden werden. Die Adresse einer Referenz kann nicht wieder verändert werden. (`int zahl = 1; int& referenz = zahl;`)
Zeiger sind Variablen, die die Speicheradresse enthalten. Die Adresse eines Zeigers kann nachträglich geändert werden. (`int zahl = 1; int* zeiger = &zahl`) Über den `&`-Operator kann auf die Adresse einer Variablen zugegriffen werden.
14. Rekursion ist ein Prinzip aus der Mathematik, welche Funktionen mit Hilfe von sich selbst definiert. Dies ist nur möglich, sofern für die Funktionen ein Abbruch-Kriterium aufweisen. In der Programmierung ist mit einer Rekursion eine Funktion gemeint, die sich selber wieder aufruft.
15. Für die rekursive Berechnung der Fibonacci-Zahlen, müssen (fast) alle Fibonacci-Zahlen mehrfach berechnet werden. Dies ist in der iterativen Berechnung nicht der Fall.
16. Klartext Buchstaben werden mit Hilfe einer Tabelle auf Geheimtext Buchstaben abgebildet. Zur Entschlüsselung wird die Verschlüsselung mit Hilfe der Tabelle in umgekehrter Richtung durchgeführt.
17. Automatischer Speicher: Lokale Variablen und Funktionsargumente.
Statischer Speicher: Globale Variablen, Variablen von namespaces, statische Elemente und Variablen
Dynamischer Speicher: Speicher, der explizit während der Laufzeit allokiert/angefordert wird. Dieser Speicher wird auch Heap genannt.
18. Durch die `break` Anweisung am Ende jedes case Labels.

19. Es muss sich um einen konstanten Wert handeln.
20. Beispiel: `sortieren(int x[2][10])`
21. Beenden von Funktionen, kann Rückgabewert übergeben, switch-case Anweisungen können verlassen werden (natürlich gemeinsam mit der Funktion, in der Sie stehen)

Versuch 3

Fragen:

什么是测试驱动开发

1. Was ist testgetriebene Entwicklung? (Fachwissen, leicht)
2. Wie ist die *gewinnerTest*-Funktion realisiert? (Verständnis, leicht)
3. Wie ist die *zugGueltig*-Funktion realisiert? (Verständnis, mittel)
4. Wie ist die *zugAusfuehren*-Funktion realisiert? (Verständnis, mittel)
5. Wie ist die *moeglicheZuege*-Funktion realisiert? (Verständnis, leicht)
6. Warum sollte Code immer gut dokumentiert sein? (Fachwissen, leicht)
7. Was ist Doxygen? (Fachwissen, leicht)
8. Wozu kann Doxygen genutzt werden? (Fachwissen, leicht)
9. Wie werden in Doxygen Kommentare gekennzeichnet? (Fachwissen, leicht)

Antworten:

通过字

人们首先定义要满足的测试, 然后实施

1. Durch Test gesteuerte Softwareentwicklung. Man definiert zunächst Tests, die erfüllt werden sollen, anschließend wird die Software so implementiert, dass sie die Tests erfüllt. Man kann die Software jederzeit gegen die Spezifikation legen. 软件以满足测试
您可以随时根据规范设置软件
2. Zählen aller Spielsteine auf dem Feld und entsprechend den Gewinner bestimmen.
3. Es wird in alle Richtungen um das gewünschte Feld gesucht, ob sich eine Richtung finden lässt, die durch einen eigenen Stein abgeschlossen ist und mindestens einen gegnerischen Stein dazwischen beinhaltet.
4. Analog zur execute_turn Funktion. Jedoch werden hier alle gültigen Richtungen betrachtet und die eingeschlossenen gegnerischen Steine werden in einer Rückwärts-Iteration umgedreht.
5. Zählt die Anzahl aller Felder, die für einen eigenen Zug genutzt werden können. Einfach turn_valid für alle Felder durchtesten und entsprechend die Anzahl der gültigen Züge mitzählen.
6. Damit man sich selber und vorallem auch andere darin schnell zurecht finden. Eine gute Dokumentation der Schnittstellen ist besonders wichtig, damit dritte die eigene Software und deren Funktionsweise und Anwendung schnell verstehen.
7. Ein Programm zur Automatischen Dokumentation in z.B. HTML-Format ausgehend von der Dokumentation im Quellcode
8. Zur Dokumentation von Softwareprojekten
9. 1. Möglichkeit:

```
/**
 * test
 **/
```

2. Möglichkeit:

```
/*!
 * test
 *!/
```

3. Möglichkeit:

```
/// test
```

4. Möglichkeit:

```
//! test
```

Versuch 4

Fragen:

1. Was versteht man unter objektorientierter Programmierung? (Fachwissen, leicht)
2. Was sind Klassen und wozu werden sie verwendet? (Fachwissen, leicht)
3. Was sind Methoden und Attribute? (Fachwissen, leicht)
4. Was ist ein Konstruktor? Was ist ein Destruktor? (Fachwissen, leicht)
5. Was sind Initialisierungslisten und wofür können diese eingesetzt werden? (Verständnis, mittel)
6. Wie werden const-Member einer Klasse initialisiert? (Fachwissen, schwierig)
7. Welche unterschiedlichen Zugriffsbeschränkungen gibt es in C++? (Fachwissen, leicht)
8. Welche Bedeutung haben die unterschiedlichen Zugriffsbeschränkungen? (Fachwissen, mittel)
9. Was sind default-Parameter und wie werden diese gekennzeichnet? (Fachwissen, leicht)
10. Was sind konstante Methoden? Was bedeutet ein const hinter einer Funktionsdeklaration? Welche Methoden können in einer konstanten Methode aufgerufen werden? (Fachwissen, mittel)
11. Was sind Include-Wächter und wofür werden diese benötigt? (Verständnis, schwierig)
12. Was ist das Prinzip einer Intervallschachtelung? (Verständnis, schwierig)
13. Wieso muss die Schrittweite so klein gewählt werden? (Anwenden, leicht)
14. Was ist der this-Zeiger? (Fachwissen, mittel)
15. Kann eine Klasse mehrere Konstruktoren und Destruktoren haben? Falls ja, wodurch unterscheiden sich diese? (Fachwissen, leicht)
16. Wann wird der Konstruktor einer Klasse aufgerufen? (Fachwissen, leicht)
17. Welche Möglichkeiten bestehen, um Attribute zu initialisieren? (Verständnis, mittel)
18. Wann muss der this-Zeiger verwendet werden? (Anwenden, schwierig)

Antworten:

1. Strukturierung der Software auf einem höheren Abstraktionsniveau. Hier stehen Objekte und Klassen sowie die Operationen und Nachrichtenaustausch zwischen Klassen und Objekten im Vordergrund. Hierüber lässt sich eine funktionale Aufteilung der Software erreichen.
2. Klassen erlauben es, benutzerdefinierte Datentypen zu erstellen. Klassen ermöglichen es, mehrere Datentypen zu einem *größeren* Datentyp zu kombinieren (Membervariablen). Darüber hinaus erlauben Klassen die Verwendung von Funktionen, die auf den Membervariablen operieren.
Klassen sollten immer verwendet werden, wenn bestimmte Funktionalitäten vorhanden sind, die einzeln gekapselt werden können oder auch, wenn bestimmte Daten gespeichert werden sollen, die nicht durch die fundamentalen Datentypen abgedeckt sind.
3. Methoden: Funktionen, die zu einer Klasse gehören. Attribute: Variablen, die zu einer Klasse gehören.
4. Konstruktor: Schablone zum Erstellen eines Objektes einer Klasse.
Destruktor: *Räumt* den Speicher beim Löschen des Objektes der Klasse wieder auf.
5. Eine Liste aller Parameter, die im Konstruktor initialisiert werden sollen. Const-Variablen können nur in Initialisierungslisten initialisiert werden. Basis-Klassen werden über Initialisierungslisten initialisiert.
6. Über die Initialisierungsliste
7. public, protected, private
8. public: auf die Member kann von überall zugegriffen werden
protected: auf die Member kann nur aus der eigenen sowie allen abgeleiteten Klassen zugegriffen werden
private: auf die Member kann nur aus der eigenen Klasse zugegriffen werden
9. Default-Parameter haben einen *Standard*-Wert und müssen nicht angegeben werden, können es jedoch. Beispiel(void multiply(double x, double factor = 2.0));
10. Konstante Methoden werden durch ein const am Ende ihrer Signatur gekennzeichnet. Konstante Methoden können lediglich aus einem Objekt lesen, jedoch nicht schreiben. Innerhalb einer konstanten Funktion können nur konstante Funktionen aufgerufen werden.
11. Include-Wächter sollen sicherstellen, dass durch Mehrfache includes eine Klasse oder ähnliches nicht mehrfach definiert wird, da dies nicht gestattet ist.
12. Durch geschickte Wahl der Schrittweite und entsprechende Reduktion der Schrittweite, kann man sich durch ein Schachteln immer näher an die exakte Lösung herantasten. Dieses Vorgehen wird häufig in der Programmierung genutzt und ist deutlich performanter als eine einfache, iterative Betrachtung mit konstanter Schrittweite
13. Dies hängt sehr vom gewählten Bereich für die Orthogonalität ab. Die Schrittweite muss so gewählt werden, dass die Iteration auf jeden Fall den Fall der orthogonalen Vektoren abdeckt.
14. Über den this-Zeiger wird das eigene Objekt angesprochen. Hierüber kann auf alle Member des Objektes zugegriffen werden. Es ist jedoch nicht notwendig, immer den this-Zeiger anzugeben, da dies vom Compiler automatisch erledigt wird.
15. Konstruktoren: Ja eine Klasse kann mehrere Konstruktoren haben. Die Konstruktoren müssen sich durch ihre Argumente unterscheiden.
Destrukturen: Eine Klasse hat immer nur einen Destruktor
16. Immer, wenn ein Objekt einer Klasse erstellt wird.
17. Initialisierungslisten oder Initialisierung innerhalb des Konstruktors
18. Bei Adressübergabe der Instanz der Klasse an andere Funktionen oder Objekte oder bei

Namenskollisionen der (Member-)Variablen.

Versuch 5

Fragen:

1. Was sind dynamische Datenstrukturen?
2. Wofür werden dynamische Datenstrukturen benötigt?
3. Nennen Sie Anwendungen von dynamischen Datenstrukturen?
4. Was ist das Prinzip von dynamischen Datenstrukturen?
5. Was ist ein Stack und nach welchem Prinzip arbeitet dieser?
6. Was ist eine Queue und nach welchem Prinzip arbeitet diese?
7. Wofür kann eine Queue eingesetzt werden?
8. Welche Arten von Listen gibt es?
9. Wo liegen die Unterschiede zwischen einer einfach und einer doppelt verketteten Liste?

Antworten:

1. Dynamische Datenstrukturen sind Datenstrukturen, die erst während der Laufzeit festgelegt werden, und auch wieder verändert werden können. Die Elemente in dynamischen Datenstrukturen werden über ihre Adresse angesprochen.
2. Statisch angelegte Datenobjekte sind während der Implementierung auf eine feste Größe festgelegt, das heißt, potentiell können diese zu wenig oder zu viel Speicherplatz bereitstellen, was ineffizient wäre. Dynamische Datenstrukturen können ihre Größe während der Laufzeit verändern und somit dynamisch auf die Anforderungen an die Datenstruktur reagieren.
3. Listen, Stacks, Queues, Implementierung von Studentendatenbank zu der Teilnehmer hinzugefügt werden können, sowie Teilnehmer entfernt werden können.
4. Der benötigte Speicher wird erst während der Laufzeit allokiert. Die einzelnen Elemente werden über einen Zeiger auf die Nachbar-Elemente miteinander verbunden.
5. Dynamische Datenstruktur, nach dem LIFO-Prinzip (Last in First out). Ein Stack arbeitet nach dem Stapelprinzip. Elemente können nur oben aufgelegt und wieder heruntergenommen werden.
6. Dynamische Datenstruktur, nach dem FIFO-Prinzip (First in First out). Eine Queue arbeitet nach dem Prinzip einer Warteschlange. Elemente können nur hinten angehängt werden und vorne wieder abgehängt werden.
7. Beispiel aus der Aufgabenstellung: Studentendatenbank.
8. Einfach verkettete Liste, doppelt verkettete Liste.
9. Einfach verkettete Liste: Jedes Element hat nur eine Referenz auf sein folgendes Element. Doppelt verkettete Liste: Jedes Element hat eine Referenz auf seinen Vorgänger und seinen Nachfolger.
Doppelt verkettete Listen, einfach von beiden Richtungen durchlaufen werden, was für das Löschen von Elementen von Vorteil sein kann. Einfach verkettete Listen können nur von vorne Durchlaufen werden.

Testatblock 2

Versuch 6

Fragen:

1. Was ist die STL?
2. Was sind Iteratoren?
3. Was sind Container?
4. Welche Funktionalitäten der STL haben Sie genutzt? Wie funktionieren diese?

Antworten:

1. Die Standard Template Library. Die STL stellt eine Vielzahl an effizienten Datenstrukturen und Algorithmen bereit. Die Implementierungen in der STL sind meist deutlich effizienter (geringe Laufzeit, geringer Speicherbedarf) als eigene Implementierungen und sollten daher immer verwendet werden.
2. Iteratoren dienen dem Zugriff auf die Elemente, die in einem Container gespeichert sind. Iteratoren können dazu genutzt werden, um durch Container zu navigieren.
3. In Containern werden Daten verwaltet, ähnlich zu den bekannten Datenstrukturen. Die Container können dynamisch wachsen. Beispiele für Container:
 - `<vector>`
 - `<list>`
 - `<deque>`
 - `<queue>`
 - `<stack>`
 - `<map>`
 - `<set>`
 - `<bitset>`
4. `std::vector`, sowie Funktionen, um Elemente im Vektor zu verwalten `std::sort` (Versuch 7)

Versuch 7

Fragen:

1. Wodurch unterscheiden sich überladenen Funktionen voneinander?
2. Kann eine Funktion durch Änderung des Rückgabewertes überladen werden?
3. Wie kann man Operatoren überladen?
4. Wozu kann die Operatorüberladung nützlich sein?
5. Welche Funktionalitäten der STL haben Sie genutzt?
6. Warum ist es für den Ausgabe-Streamoperator nicht möglich, diesen innerhalb der Klasse Student zu überladen? Was ist das Problem bei der Überladung des Ausgabe-Operators?

Antworten:

1. Die Funktionen müssen sich durch die Anzahl der Parameter oder den Typ der Parameter unterscheiden. Eine Unterscheidung anhand des Rückgabe-Typs ist nicht möglich.
2. Nein.
3. Unärer Operator (innerhalb der Klasse):

Rückgabetyt Klassenname::operator <op>() { Code }

Beispiel:

```
komplexeZahl komplexeZahl::operator !()
{
    return this->invertieren();
}
```

Binärer Operator (innerhalb der Klasse):

Rückgabetyt Klassenname::operator <op>(Typ operand) { Code }

Beispiel:

```
komplexeZahl komplexeZahl::operator * (const double& faktor)
{
    return this->skalieren(faktor);
}
```

Anmerkung: Die Operatoren können auch außerhalb der Klasse überladen werden, dann muss als lhs-Argument immer der entsprechende Klassentyp erscheinen, das rhs Argument ist dann analog zu dem Beispiel oben definiert.

4. Einfaches Vergleichen von Klassenobjekten, nach einem benutzerdefiniertem Kriterium. Oder zur gezielten Ausgabe bestimmter Informationen der Klasse.
5. std::sort
6. Für die Überladung des Ausgabe-Operators ist das lhs Argument std::cout ein Objekt der Klasse ostream. Daher kann der Operator nicht innerhalb der Student-Klasse überladen werden. Da der Quellcode der Klasse std::cout nicht verfügbar ist, muss der Ausgabe-Operator außerhalb der Klasse std::cout überladen werden.

Versuch 8

Fragen:

1. Was versteht man unter Polymorphie?
2. Was versteht man unter später Bindung?
3. Was versteht man unter Vererbung?
4. Was sind virtuelle Methoden und wie werden diese deklariert?
5. Was sind abstrakte Klassen?
6. Wodurch sind abstrakte Klassen gekennzeichnet?
7. Können Objekte von abstrakten Klassen erzeugt werden?
8. Was ist eine rein virtuelle Methode?
9. Warum sollte der Destruktor immer virtuell sein?
10. Erklären sie die Begriffe public, private, protected in Bezug auf die Vererbung.
11. Warum sind Initialisierungslisten für Vererbungshierarchien wichtig?

Antworten:

1. Verschiedene Objekte können sich beim Aufruf derselben Methode unterschiedlich verhalten.
2. Methoden abgeleiteter Klassen werden aufgerufen, auch wenn der Zeiger nur vom Typ der Basisklasse ist. Erst zur Laufzeit wird entschieden, welche konkrete Methode dem Aufruf einer Operation zugeordnet wird. Dies wird durch virtuelle Methoden erreicht.
3. Über die Vererbung lassen sich Beziehungen zwischen Klassen herstellen. Es gibt Basisklassen und Unterklassen, siehe Beispiel mit den Klassen Mitarbeiter, Manager, Angestellter.
Die Basisklasse stellt hierbei eine gewissen Grundfunktionalität zur Verfügung, die für alle abgeleiteten Klassen sinnvoll sind. In den abgeleiteten Klassen kann dann weitere Funktionalität implementiert werden, die nur hier benötigt wird.
4. Virtuelle Methoden sind spät gebunden, das heißt es wird zur Laufzeit entschieden, welche Implementierung ausgeführt wird. Abgeleitete Klassen können eine neue Implementierung für die virtuelle Methode hinzufügen, müssen dies aber nicht. Virtuelle Methoden werden durch das Schlüsselwort `virtual` vor dem Rückgabotyp der Funktion deklariert.
Beispiel: `virtual void print();`
5. Abstrakte Klassen sind Klassen, die mindestens eine rein virtuelle Funktion aufweisen. Es können keine Objekte von abstrakten Klassen erstellt werden.
6. Durch mindestens eine rein virtuelle Funktion.
7. Nein.
8. Eine rein virtuelle Methode ist durch ein `= 0` am Ende ihrer Signatur gekennzeichnet. Rein virtuelle Methoden benötigen in der Klasse in der sie gekennzeichnet sind keine Implementierung.
9. Damit beim Löschen der Basisklasse auch der entsprechende Destruktor der abgeleiteten Klasse aufgerufen wird und somit keine Memory-Leaks entstehen.
10. `public`: Member stehen überall zur Verfügung.
`protected`: Member stehen nur in der Klasse, sowie allen abgeleiteten Klassen zur Verfügung.
`private`: Member stehen nur in der Klasse in der sie deklariert sind zur Verfügung.
11. Um auf die Basisklasse initialisieren zu können.

Versuch 9

Fragen:

1. Wie funktioniert die Kommunikation zwischen GUI Elementen und Funktionen in Qt?
2. Was sind statische Funktionen?
3. Wofür können statische Funktionen eingesetzt werden?
4. Was sind statische Attribute?
5. Wofür können statische Attribute eingesetzt werden?
6. Wie greifen Sie auf Elemente der grafischen Oberfläche zu?
7. Erklären Sie das Zeichnen der Karte.
8. Was ist das besondere an der `exec()` Funktion eines Dialogs?
9. Wie haben Sie sichergestellt, dass Ihre Implementierung der Map den Anforderungen entspricht, die die mitgelieferte Dijkstra-Implementierung benötigt?
10. Wenn Sie keine Konsole nutzen wollen, welche Möglichkeiten bestehen, Testergebnisse auszugeben?

Antworten:

In diesem Versuch geht es weniger um die Abfrage von theoretischem Wissen. Es soll vielmehr sichergestellt werden, dass die geforderten Konzepte verstanden wurden. Die Destruktoren sollten sich im Detail angesehen werden, daran lässt sich viel erkennen.

1. Mit Hilfe des Signal-Slot-Konzeptes. Die spezifizierte Slot Funktion wird jedes mal aufgerufen, wenn das verbundene Signal emittiert wird.
2. Statische Funktionen sind Funktionen, die auch ohne ein Objekt einer Klasse aufgerufen werden können. Statische Funktionen können nur auf statische Member der Klasse zugreifen.
3. Beispiel aus dem Praktikum: Qstring, Dijkstra
Immer wenn bestimmte Funktionalität auch sinnvoll ohne ein Objekt einer Klasse implementiert werden kann, können hierfür statische Funktionen genutzt werden.
4. Variablen, die mit dem zusätzlichen Schlüsselwort static gekennzeichnet sind. Statische Attribute sind in allen Instanzen der Klasse identisch.
5. Zum Beispiel um erzeugte Objekte einer Klasse zu zählen.
6. Über die entsprechenden Funktionen, die QT zur Verfügung stellt.
7. Implementierung erklären lassen.
8. ?
9. Dadurch, dass die entsprechenden, gegebenen Schnittstellen implementiert und verwendet wurden.
10. Es können auch Ausgaben in z.B. MessageBoxes geschrieben werden.